

Implementing Nelder-Mead method

1. Implement the two and three dimensional versions of Nelder-Mead method. Use a programming language of your choice.

One step beyond: Why fix the dimension? Surely you can pass the dimension as a parameter or better still compute it on the fly.

Comparing Nelder-Mead with gradient descent methods

2. Qualitatively compare Nelder-Mead method with the methods you have implemented in HW GD.2 (GD, Polyak GD, Nesterov GD, AdaGrad GD, Newton, BFGS) on

$$f(x, y, z) = (x - z)^2 + (2y + z)^2 + (4x - 2y + z)^2 + x + y,$$

$$f(x, y, z) = (x - 1)^2 + (y - 1)^2 + 100(y - x^2)^2 + 100(z - y^2)^2,$$

and

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2.$$

These functions are taken from M2 GD.2/6a,b,c. For the Nelder-Mead method choose starting samples of different diameters, one of the starting sample points should match the one given in the original homework.

One step beyond: Qualitatively compare?? Think of all possible aspects of comparison. The word *all* in the previous sentence is maybe too ambitious, *several* is a more appropriate choice.

Black box optimization

Let `xxxxxxxx` be your student Id. The three functions

$$f_{xxxxxxxx,i} : \mathbb{R}^3 \rightarrow \mathbb{R},$$

$i \in \{1, 2, 3\}$ are given in a black-box setting.

The archive `hw4_1.executables.zip` (contained in the Part 4 HW1 dropbox) contains three (equivalent) command line executables `hw4_1_linux`, `hw4_1_mac`, and `hw4_1_win.exe`, running on three different OS families. Each executable expects five parameters on the standard input and, after a time consuming computation, outputs a real number to the standard output. Below is a sample call.

```
$ ./hw4_1_mac xxxxxxxx i 3.17 0.71 -1.55
```

Your student Id is the first parameter `xxxxxxxx`, the second parameter is an integer $i \in \{1, 2, 3\}$, and the remaining three parameters are real numbers.

Try to test the appropriate executable as fast as possible. Please report any problems should the above program not work as intended.

3. Find minima of functions

$$f_{\text{XXXXXXXX}}, 1, \quad f_{\text{XXXXXXXX}}, 2, \quad f_{\text{XXXXXXXX}}, 3.$$

Use sufficiently high precision (twelve significant digits or more). Pay attention that the results should not be the same as those obtained by previous generations of students.

These problems are in theory unconstrained, you are guaranteed that none of the calls results in an overflow if real parameters lie in $[-10, 10]$.

One step beyond: How would one use a gradient-descent based method in such a case. Which one is best suitable. Can you beat Nelder-Mead?

A toy solution to a linear problem

Observe the following linear problem.

$$\begin{aligned} \max \quad & x_1 + 3x_2 + 4x_3 \\ & x_1 - x_2 + x_3 \leq 5 \\ & x_1 + x_2 - x_3 \leq 7 \\ & -x_1 + x_2 + x_3 \leq 10 \\ & x_1 + 2x_2 + 3x_3 \leq 8 \\ & 3x_1 + 2x_2 + x_3 \leq 6 \\ & x_1, x_2, x_3 \geq 0. \end{aligned}$$

You have altogether eight linear inequalities (here we treat nonnegativity constraints in the same way as the other ones). In principle, picking an arbitrary triple Θ of constraints, and treating them as equalities, yields a unique solution $x^\Theta \in \mathbb{R}^3$. It may be feasible, satisfying the remaining constraints, or not.

4. Find the optimal solution to the above LP by examining vectors x^Θ obtained over all possible triples Θ of constraints.

One step beyond: How does this approach generalize if we increase the number of variables and the number of constraints?

Bottom line

Upload your solution in a single .zip archive which contains the source-code (I would be most pleased with a Jupyter notebook in Python - nevertheless you can use any programming platform/language according to your personal preferences) and a .pdf.